

NumPy

- Jake VetaerPlas. 2016. *Python Data Science Hetabook: Essential Tools for Working with Data*. O'Reilly Media, Inc.
- 1. kapitulua - NumPy-rako sarrera
- <https://github.com/jakevdp/PythonDataScienceHetabook>
(<https://github.com/jakevdp/PythonDataScienceHetabook>)

NumPy-k honako hauek eskaintzen ditu:

- Memorian eraginkorrak diren N dimentsioko arrayak, eragiketa azkar eta bektorizatuekin.
- Aljebra linealerako, estatistikarako, ausazko zenbakiak sortzeko eta abarretarako funtzio matematikoak.
- Forma desberdin baina bateragarriak dituzten arrayen arteko elementuz elementuko eragiketak (**broadcasting** edo hedapena).
- Arrayen indexazio aurreratua
- Python-eko beste liburutegi zientifiko askoren oinarria (SciPy, Pandas, Matplotlib, Scikit-learn)

```
In [1]: import numpy as np
        np.__version__
```

```
Out[1]: '1.26.4'
```

```
In [2]: # sakatu TAB NumPy-ren izen-espazioa ikusteko
        #np.
```

Datu motak

NumPy-k zenbakizko arrayetarako inplementazio alternatibo bat eskaintzen du, eta datuetan oinarritutako kalkuluaren errendimendua hobetzen du Python-en berezko zerrendekin alderatuta.

Python-eko zenbaki osoak eta C-ko zenbaki osoak

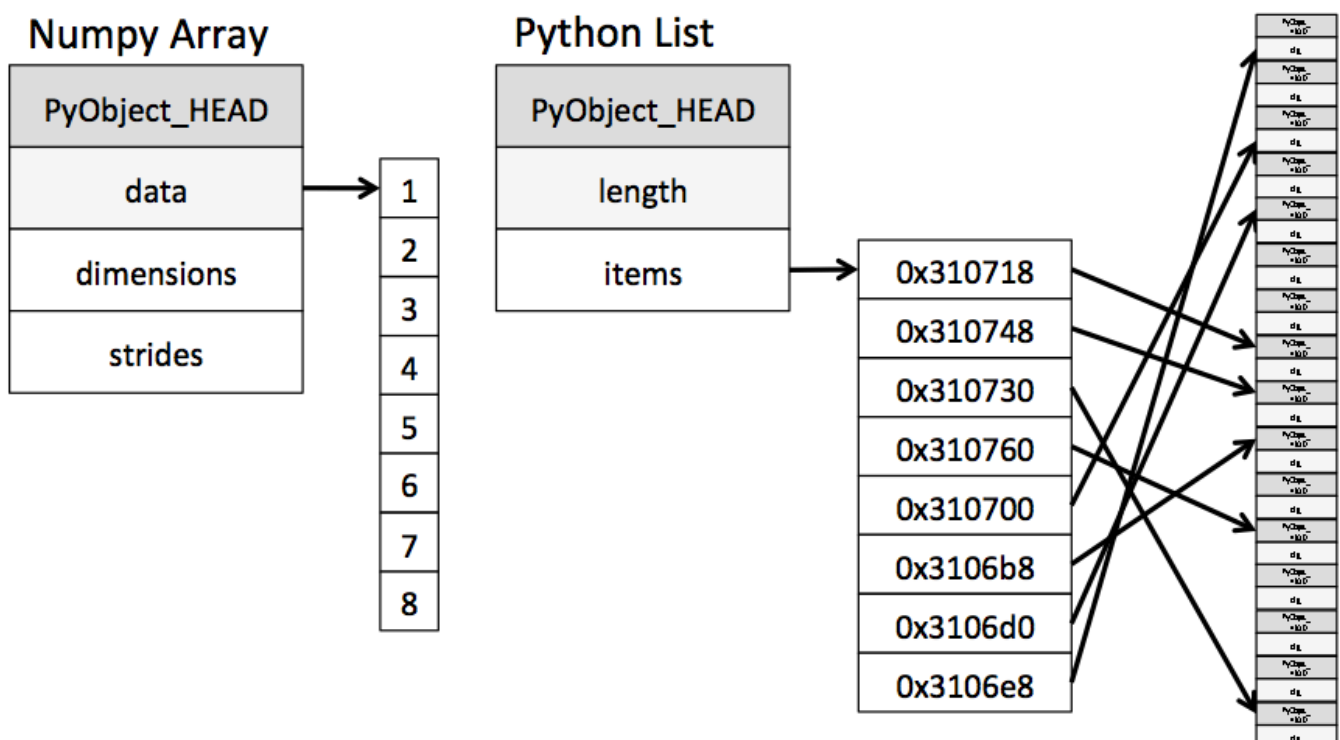
- Python-eko `int` balioak objektu *konplexuak* dira (C-n idatziak)
 - Dinamikoki tipatutako lengoaia
 - Zenbaki osoen aritmetikan ia zehaztasun mugagabea

```
struct _longobject {  
    long ob_refcnt;           # reference count  
    PyTypeObject *ob_type;    # type of the variable  
    size_t ob_size;           # size of the following data members  
    long ob_digit[1];         # integer value encoded into a long array  
};
```

- C lengoaiako zenbaki osoak (`char`, `short`, `int`, `long`, `long...`) memoriako kokapen baterako erreferentzia soilak dira; kokapen horretako byteek zenbaki oso bat kodetzen dute.

Python-eko zerrendak eta NumPy arrayak

- Python-eko `list` zerrendak objektu *konplexuak* dira (`int` balioak baino askoz konplexuagoak)
 - `list` zerrendak heterogeneoak dira
 - Objektu mota (eta tamaina) desberdinak
 - `list` zerrendek objektu bakoitzerako erreferentziak dituen array bat daukate
 - Erakusle-bloke baterako erakusle bat daukate, eta blokeko erakusle bakoitzak Python objektu bat seinalatzen du
- NumPy-ren array *estandarrek* homogeneoak dira.
 - Datu-bloke jarraitu baterako erakusle bakarra daukate.



NumPy arrayak

Python-eko zerrendetatik arrayak sortzea

- `np.array(some_list)` → array (homogeneo) bat sortu
- `np.array(some_list, dtype=<data type>)` → adierazitako motako array bat sortu

```
In [3]: np.array([1, 4, 2, 5, 3])
```

```
Out[3]: array([1, 4, 2, 5, 3])
```

```
In [4]: np.array([1, 4, 2, 5, 3], dtype='float32')
```

```
Out[4]: array([1., 4., 2., 5., 3.], dtype=float32)
```

Motak bat ez badatoz, NumPy-k goranzko bihurketa (*upcast*) egingo du, ahal izanez gero:

```
In [5]: np.array([1, 4, 2, 5.9, 3])
```

```
Out[5]: array([1. , 4. , 2. , 5.9, 3. ])
```

Habiaratutako zerrendek dimentsio anitzeko arrayak sortzen dituzte

```
In [6]: np.array([[ 0,  1,  2,  3], [10, 11, 12, 13], [20, 21, 22, 23]])
```

```
Out[6]: array([[ 0,  1,  2,  3],
               [10, 11, 12, 13],
               [20, 21, 22, 23]])
```

NumPy-ren datu mota estetaarrak

Array bat eraikitzean, datu motaren formatu argumentua (`dtype`) honela zehaztu daiteke:

- karaktere-katea → `dtype='float32'`
- NumPy objektua → `dtype=np.float32`

```
In [7]: np.array([1, 4, 2, 5, 3], dtype='float32')
```

```
Out[7]: array([1., 4., 2., 5., 3.], dtype=float32)
```

```
In [8]: np.array([1, 4, 2, 5, 3], dtype=np.float64)
```

```
Out[8]: array([1., 4., 2., 5., 3.])
```

Datu mota	Deskribapena
bool_	Boolearra (True edo False), byte batean gordea
int_	Defektuzko zenbaki osoa (C-ko long; normalean int64 edo int32)
intc	C-ko int (normalean int32 edo int64)
intp	Indexatzeko erabiltzen den osokoa (C-ko ssize_t; normalean int32 edo int64)
int8	Byte [-128,127]
int16	Osoa (-)[-32768,32767]
int32	Osokoa [-2147483648,2147483647]
int64	Osokoa [-9223372036854775808,9223372036854775807]
uint8	Zeinurik gabeko osoa [0,255]
uint16	Zeinurik gabeko osoa [0,65535]
uint32	Zeinurik gabeko osoa [0,4294967295]
uint64	Zeinurik gabeko osoa [0,18446744073709551615]
float_	float64 motako formatuaren laburdura.
float16	Doitasun erdiko erreala: zeinu-bita, 5 bit berretzailean, 10 bit mantisan
float32	Doitasun bakuneko erreala: zeinu-bita, 8 bit berretzailean, 23 bit mantisan
float64	Doitasun bikoitzeko erreala: zeinu-bita, 11 bit berretzailean, 52 bit mantisan
complex_	complex128 motako formatuaren laburdura.
complex64	Zenbaki konplexua, bi 32-bitetako zenbaki errealeen bidez adierazia
complex128	Zenbaki konplexua, bi 64-bitetako zenbaki errealeen bidez adierazia

Arrayak hutsetik sortzea

dtype -ren balio lehenetsiak (gehienetan):

- Osokoak → int64
- Errealak (koma higikorra) → float64

- np.zeros(10) → zeroz betetako 10 luzerako array bat

```
In [9]: np.zeros(10)
```

```
Out[9]: array([0., 0., 0., 0., 0., 0., 0., 0., 0., 0.])
```

```
In [10]: np.zeros(10, dtype='int64')
```

```
Out[10]: array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0])
```

```
In [11]: np.zeros(10, dtype='float32')
```

```
Out[11]: array([0., 0., 0., 0., 0., 0., 0., 0., 0., 0.], dtype=float32)
```

```
In [12]: np.zeros(10, dtype='int32')
```

```
Out[12]: array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0], dtype=int32)
```

- `np.eye(3)` → 3x3 identitate-matrize bat

```
In [13]: np.eye(3)
```

```
Out[13]: array([[1., 0., 0.],  
               [0., 1., 0.],  
               [0., 0., 1.]])
```

- `np.ones((3,5))` → batekoz betetako 3x5 array bat

```
In [14]: np.ones((3,5))
```

```
Out[14]: array([[1., 1., 1., 1., 1.],
                [1., 1., 1., 1., 1.],
                [1., 1., 1., 1., 1.]])
```

- `np.full((3,5), 1.8)` → 3x5 array bat, honako balio honekin betea: 1.8

```
In [15]: np.full((3,5), 1.8)
```

```
Out[15]: array([[1.8, 1.8, 1.8, 1.8, 1.8],
                [1.8, 1.8, 1.8, 1.8, 1.8],
                [1.8, 1.8, 1.8, 1.8, 1.8]])
```

- `np.empty((3,5))` → *hasieratu gabeko* 3x5 array bat

```
In [16]: np.empty((3,5))
```

```
Out[16]: array([[1.8, 1.8, 1.8, 1.8, 1.8],
                [1.8, 1.8, 1.8, 1.8, 1.8],
                [1.8, 1.8, 1.8, 1.8, 1.8]])
```

- `np.arange(4, 20, 2)` → array bat, [4, 20) tarteko sekuentzia lineal batekin, 2 urratsarekin.

```
In [17]: np.arange(4, 20, 2)
```

```
Out[17]: array([ 4,  6,  8, 10, 12, 14, 16, 18])
```

- `np.linspace(1, 7, 5)` → array bat, [1, 7] tartean uniformeki banatutako bost baliorekin:

```
In [18]: np.linspace(1, 7, 5)
```

```
Out[18]: array([1. , 2.5, 4. , 5.5, 7. ])
```

Sasi-ausazko zenbakiak

- `np.random.random((3,5))` → 3x5 array bat, $[0, 1)$ tarte erdi-irekian uniformeki banatutako ausazko balioekin (behin baino gehiagotan exekutatu):

```
In [19]: np.random.random((3,5))
```

```
Out[19]: array([[0.03382434, 0.26481628, 0.131467  , 0.01419255, 0.72271835],
                [0.46268739, 0.49259168, 0.56097584, 0.81764734, 0.45471118],
                [0.43213466, 0.15200314, 0.16656268, 0.30978019, 0.38375528]])
```

- `np.random.seed(int)` → ausazko zenbakiak sortzeko hazia ezartzen du. **Erreproduzigarritasuna eskaintzen du** (behin baino gehiagotan exekutatu).

```
In [20]: np.random.seed(43)
         np.random.random((3,5))
```

```
Out[20]: array([[0.11505457, 0.60906654, 0.13339096, 0.24058962, 0.32713906],
                [0.85913749, 0.66609021, 0.54116221, 0.02901382, 0.7337483  ],
                [0.39495002, 0.80204712, 0.25442113, 0.05688494, 0.86664864]])
```

- `np.random.normal(10, 2, (3,5))` → 3x5 array bat, banaketa normaleko ausazko balioekin, $\mu = 10$ eta $\sigma = 2$

```
In [21]: np.random.normal(10, 2, (3, 5))
```

```
Out[21]: array([[ 9.06239973,  7.247008  ,  8.96229899,  9.54920845, 10.41643253],
                [10.40961276,  9.83619328, 10.67376611,  9.85909104, 11.18066954],
                [ 8.80587096, 10.48544469,  9.38402231, 11.21093121,  7.00980231]])
```

- `np.random.randint(-7, 3, (3, 5))` → 3x5 array bat, $[-7, 3)$ tarte erdi-irekiko ausazko zenbaki osokoekin:

```
In [22]: np.random.randint(-7, 3, (3, 5))
```

```
Out[22]: array([[ -2,  -2,  -2,   1,   0],
                [-4,  -1,  -4,  -5,  -3],
                [-5,   0,  -4,  -1,   1]])
```

Konputazio paraleloa eta ausazko zenbakien sorgailuak (RNG)

Programa bat paraleloan exekutatzen denean (hainbat worker erabiliz), normalean worker bakoitzak bere ausazko zenbakiak sortzea nahi da.

- Eman worker guztiei **hazi bera** → zenbaki-sekuentzia berbera
- Eman worker bakoitzari **hazi desberdina** → sekuentzien gainjartzea eta korrelazio estatistikoak

Irtenbidea worker bakoitzari RNG **independente** bat ematea da.

1. `parent_RNG = np.random.default_rng(seed)` → RNG (guraso) objektu bat itzultzen du
2. `children_RNG = parent_RNG.spawn(size)` → matematikoki independenteak diren n RNG sortzen ditu
3. erabili `children_RNG[i]`

Honako hau lortzen da:

- Independentzia estatistikoa
- Erreproduzigarritasun osoa

NumPy arrayei buruz gehiago

- Arrayhauen formakn dimentsioak
- Arrayhauen formakn atributuak
- Arrayhauen formakn indexazioa
- Arrayhauen formakn zatitzea
- Arrayen forma aldatzea
- Arrayak kateatzea eta zatitzea

Arrayhauen formakn dimentsioak

NumPy arrayek edozein dimentsio kopuru izan dezakete:

- 0 dimentsioko arrayak → **eskalarrak** edo **0 mailako tentsoreak**
- 1 dimentsioko arrayak → **bektoreak** edo **1 mailako tentsoreak**
- 2 dimentsioko arrayak → **matrizeak** edo **2 mailako tentsoreak**
- 3 dimentsioko arrayak → **tentsoreak** edo **3 mailako tentsoreak**
- ...
- N dimentsioko arrayak → **tentsoreak** edo **N mailako tentsoreak**

```
In [23]: np.random.randint(1,10,(2,2,2,3,6))
```

```
Out[23]: array([[[[4, 7, 8, 5, 6, 4],
                  [6, 4, 9, 4, 6, 2],
                  [7, 1, 1, 8, 2, 4]],

                [[2, 6, 9, 1, 4, 7],
                  [5, 7, 8, 5, 6, 9],
                  [5, 8, 5, 3, 7, 8]]],

               [[9, 9, 2, 9, 6, 6],
                [9, 7, 9, 4, 5, 3],
                [4, 2, 5, 3, 3, 2]],

               [[5, 6, 6, 1, 4, 5],
                [8, 2, 3, 8, 4, 2],
                [2, 4, 7, 6, 9, 1]]]],

            [[[6, 1, 9, 7, 8, 9],
              [9, 8, 2, 5, 2, 7],
              [4, 1, 2, 8, 9, 4]],

             [[3, 1, 4, 3, 1, 3],
              [7, 8, 5, 5, 7, 1],
              [5, 7, 1, 5, 7, 4]]],

            [[7, 8, 9, 2, 5, 6],
             [2, 9, 1, 3, 3, 8],
             [7, 6, 1, 5, 1, 9]],

            [[9, 2, 9, 2, 7, 1],
             [3, 2, 6, 1, 6, 6],
             [9, 4, 3, 1, 6, 6]]]])
```

Arrayhuen formakn atributuak

x NumPy array bat emanda:

- `x.ndim` → dimentsio kopurua
- `x.shape` → `x.ndim` tamainako n-kotea (tupla), dimentsio bakoitzaren tamaina jasotzen duena
- `x.size` → arrayaren guztizko tamaina
- `x.dtype` → arrayaren datu mota

```
In [24]: x = np.random.rand(3,4)
print(x)
print(f'{x.ndim=} {x.shape=} {x.size=} {x.dtype=}')

[[0.9973726  0.48601552 0.32129159 0.973439  ]
 [0.99871854 0.4452131  0.0447457  0.44706112]
 [0.7101748  0.62607133 0.67064977 0.97191886]]
x.ndim=2 x.shape=(3, 4) x.size=12 x.dtype=dtype('float64')
```

Arrayen indexazioa

Dimentsio bakarreko arrayak Pythoneko zerrendak bezala indexatzen dira:

```
In [25]: x = np.arange(100,110)
x[3]
```

Out[25]: 103

Indize negatiboak erabil daitezke, Python zerrendetan bezala:

```
In [26]: x[-1],x[-10]
```

Out[26]: (109, 100)

Dimentsio anitzeko arrayak komaz bereizitako indize-tupla baten bidez indexatzen dira

```
In [27]: x = np.random.rand(3,4)
print(x)
print(x[0,3], x[-1,0])

[[0.66197124 0.0914502  0.58973101 0.21158101]
 [0.79507563 0.35030598 0.50243648 0.70353915]
 [0.17162768 0.60105944 0.16003461 0.8265784  ]]
0.21158100640626754 0.17162767591793848
```

Balioak aurreko edozein indexazio-notazio erabiliz alda daitezke.

```
In [28]: x = np.arange(10)
print(x)
x[-1] = 20
print(x)
```

```
[0 1 2 3 4 5 6 7 8 9]
[ 0  1  2  3  4  5  6  7  8 20]
```

NumPy arrayek mota finkoak dituzte. `int` motako array baten elementu bati balio erreal bat esleitzen bazaio, balioa trunkatu egiten da.

```
In [29]: print(x)
x[-1] = 17.321
print(x)
```

```
[ 0  1  2  3  4  5  6  7  8 20]
[ 0  1  2  3  4  5  6  7  8 17]
```

Arrayen zatitzea (*slice notazioa*)

NumPy-ren slice notazioak Python-en zerrenden sintaxi bera du

- `x[start:stop:step]`
- Ez badira adierazten, balio lehenetsiak erabiltzen dira:
 - `start=0`
 - `stop=size_of_dimension`
 - `step=1`

```
In [30]: x = np.arange(100,110)
print(f'{x[:5]=}')
print(f'{x[5:]=}')
print(f'{x[4:7]=}')
print(f'{x[::2]=}')
print(f'{x[::-1]=}')
print(f'{x[5::-2]=}')
```

```
x[:5]=array([100, 101, 102, 103, 104])
x[5:]=array([105, 106, 107, 108, 109])
x[4:7]=array([104, 105, 106])
x[::2]=array([100, 102, 104, 106, 108])
x[::-1]=array([109, 108, 107, 106, 105, 104, 103, 102, 101, 100])
x[5::-2]=array([105, 103, 101])
```

```
In [31]: x = np.random.randint(1,10,(3,5))
print(f'{x=}')
print(f'{x[1,:]=}')
print(f'{x[1]=}') # baliokidea
print(f'{x[:,1]=}')
print(f'{x[:2,3:]=}')
print(f'{x[0,:-1]=}')
print(f'{x[:-1,1]=}')
```

```
x=array([[8, 2, 6, 1, 5],
        [3, 7, 4, 6, 8],
        [9, 9, 3, 9, 9]])
x[1,:]=array([3, 7, 4, 6, 8])
x[1]=array([3, 7, 4, 6, 8])
x[:,1]=array([2, 7, 9])
x[:2,3:]=array([[1, 5],
               [6, 8]])
x[0,:-1]=array([5, 1, 6, 2, 8])
x[:-1,1]=array([9, 7, 2])
```

Python zerrenden sliceek ez bezala, arrayen sliceek **bistak** itzultzen dituzte, ez kopiak:

```
In [32]: x = np.random.rand(3,5)
print(f'{x=}')
y = x[1,:2]
print(f'{y=}')
print('-'*40)
y[0] = 0
print(f'{y=}')
print(f'{x=}')

```

```
x=array([[0.73506435, 0.6940475 , 0.05724693, 0.47188981, 0.57466076],
        [0.2819044 , 0.53691866, 0.85055804, 0.0388812 , 0.81181727],
        [0.74221649, 0.53472615, 0.17077938, 0.71274351, 0.40038822]])
y=array([0.2819044 , 0.85055804, 0.81181727])
-----
y=array([0.          , 0.85055804, 0.81181727])
x=array([[0.73506435, 0.6940475 , 0.05724693, 0.47188981, 0.57466076],
        [0.          , 0.53691866, 0.85055804, 0.0388812 , 0.81181727],
        [0.74221649, 0.53472615, 0.17077938, 0.71274351, 0.40038822]])
```

Arrayen edo azpiarrayen (sliceen) kopia esplizituak sor daitezke: `x.copy()`

```
In [33]: x = np.random.rand(3,5)
print(f'{x=}')
y = x[1,:2].copy()
print(f'{y=}')
print('-'*40)
y[0] = 0
print(f'{y=}')
print(f'{x=}')

x=array([[0.49829084, 0.11262131, 0.22716925, 0.60550357, 0.69519328],
        [0.87250491, 0.02535821, 0.07351588, 0.19001353, 0.36446791],
        [0.5241581 , 0.5459378 , 0.24354811, 0.89363123, 0.90270436]])
y=array([0.87250491, 0.07351588, 0.36446791])
-----
y=array([0.          , 0.07351588, 0.36446791])
x=array([[0.49829084, 0.11262131, 0.22716925, 0.60550357, 0.69519328],
        [0.87250491, 0.02535821, 0.07351588, 0.19001353, 0.36446791],
        [0.5241581 , 0.5459378 , 0.24354811, 0.89363123, 0.90270436]])
```

Arrayen forma aldatzea

- `reshape(a, newshape)` → forma eraldatuta duen array originalaren **bista** bat
- `a.reshape(newshape)` → forma eraldatuta duen array originalaren **bista** bat

```
In [34]: np.reshape(np.arange(10), (2, 5))
```

```
Out[34]: array([[0, 1, 2, 3, 4],
               [5, 6, 7, 8, 9]])
```

```
In [35]: np.arange(10).reshape((2, 5))
```

```
Out[35]: array([[0, 1, 2, 3, 4],
               [5, 6, 7, 8, 9]])
```

```
In [36]: a = np.arange(10)
b = a.reshape((2,5))
c = b.reshape((1,10))
b[0,0] = -8
print(f'{a=}\n{b=}\n{c=}')

a=array([-8,  1,  2,  3,  4,  5,  6,  7,  8,  9])
b=array([[ -8,  1,  2,  3,  4],
        [ 5,  6,  7,  8,  9]])
c=array([[ -8,  1,  2,  3,  4,  5,  6,  7,  8,  9]])
```

`np.newaxis` , array slicing-a erabiltzen denean, dimentsio berri bat gehitzen du (`reshape` egiten du):

```
In [37]: from numpy import newaxis
a = np.arange(4)      # bektorea
b = a[newaxis,:]      # row bektorea a.reshape((1,4))
c = a[:,newaxis]      # column bektorea a.reshape((4,1))
a[0] = -8
print(f'{a=}\n{b=}\n{c=}')

a=array([-8,  1,  2,  3])
b=array([[ -8,  1,  2,  3]])
c=array([[ -8],
        [  1],
        [  2],
        [  3]])
```

Arrayak kateatzen eta zatitzen

- Arrayen **kateatzea** → hainbat array bakar batean konbinatzea
 - `np.concatenate` , `np.vstack` eta `np.hstack`
- Arrayen **zatitzea** → array bakar bat hainbat arraytan banatzea
 - `np.split` , `np.vsplit` eta `np.hsplit`
- `concatenate((a1, a2, ...), axis=0, out=None, dtype=None, casting="same_kind")`
 - `(a1, a2, ...)` → array-sekuentzia
 - `axis` → arrayak zein ardatzetan elkartuko diren
 - arrayek `axis` parametroari dagokion dimentsioa izan behar dute
 - arrayek forma bera izan behar dute, `axis` parametroari dagokion dimentsioan izan ezik

```
In [38]: a = np.arange(0,4)
b = np.arange(4,8)
c = np.arange(8,12)
d = np.concatenate((a,b,c))
print(f'{a=}\n{b=}\n{c=}\n{d=}')

a=array([0, 1, 2, 3])
b=array([4, 5, 6, 7])
c=array([ 8,  9, 10, 11])
d=array([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11])
```

```
In [39]: # ERRORREA: a, b eta c arrayek ez dute 1 dimentsioa
#np.concatenate((a,b,c), axis=1)
```

```
In [40]: a2,b2,c2 = a[newaxis,:],b[newaxis:],c[newaxis:]
print(f'{a2=}\n{b2=}\n{c2=}')

a2=array([[0, 1, 2, 3]])
b2=array([[4, 5, 6, 7]])
c2=array([[ 8,  9, 10, 11]])
```

```
In [41]: np.concatenate((a2,b2,c2))
```

```
Out[41]: array([[ 0,  1,  2,  3],
               [ 4,  5,  6,  7],
               [ 8,  9, 10, 11]])
```

```
In [42]: np.concatenate((a2,b2,c2), axis=0)
```

```
Out[42]: array([[ 0,  1,  2,  3],
               [ 4,  5,  6,  7],
               [ 8,  9, 10, 11]])
```

```
In [43]: np.concatenate((a2,b2,c2), axis=1)
```

```
Out[43]: array([[ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11]])
```

- `vstack` eta `hstack` - 2 (edo gehiago) dimentsioko arrayak
 - `vstack((a1, a2, ...)) == concatenate((a1, a2, ...), axis=0)`
 - `hstack((a1, a2, ...)) == concatenate((a1, a2, ...), axis=1)`

```
In [44]: a = np.arange(0,8).reshape((2,4))
b = np.arange(8,16).reshape((2,4))
c = np.hstack((a,b))
d = np.concatenate((a,b), axis=1)
e = np.vstack((a,b))
f = np.concatenate((a,b), axis=0)
print(f'{a=}\n{b=}\n{c=}\n{d=}\n{e=}\n{f=}')
```

```
a=array([[0, 1, 2, 3],
        [4, 5, 6, 7]])
b=array([[ 8,  9, 10, 11],
        [12, 13, 14, 15]])
c=array([[ 0,  1,  2,  3,  8,  9, 10, 11],
        [ 4,  5,  6,  7, 12, 13, 14, 15]])
d=array([[ 0,  1,  2,  3,  8,  9, 10, 11],
        [ 4,  5,  6,  7, 12, 13, 14, 15]])
e=array([[ 0,  1,  2,  3],
        [ 4,  5,  6,  7],
        [ 8,  9, 10, 11],
        [12, 13, 14, 15]])
f=array([[ 0,  1,  2,  3],
        [ 4,  5,  6,  7],
        [ 8,  9, 10, 11],
        [12, 13, 14, 15]])
```

- `vstack` eta `hstack` - 1 dimentsioko arrayak

- `vstack((a1, a2, ...)) == concatenate((a1[newaxis,:], a2[newaxis,:], ...))`
- `hstack((a1, a2, ...)) == concatenate((a1, a2, ...))`

```
In [45]: a = np.arange(0,4)
b = np.arange(4,8)
c = np.hstack((a,b))
d = np.concatenate((a,b))
e = np.vstack((a,b))
f = np.concatenate((a[newaxis,:],b[newaxis,:]))
print(f'{a=}\n{b=}\n{c=}\n{d=}\n{e=}\n{f=}')
```

```
a=array([0, 1, 2, 3])
b=array([4, 5, 6, 7])
c=array([0, 1, 2, 3, 4, 5, 6, 7])
d=array([0, 1, 2, 3, 4, 5, 6, 7])
e=array([[0, 1, 2, 3],
        [4, 5, 6, 7]])
f=array([[0, 1, 2, 3],
        [4, 5, 6, 7]])
```


- `split(a, split_points, axis=0)`
 - `a` → zatitu beharreko arraya
 - `split_points`
 - N (int) → arraya N array berdinetan banatzen da (edo huts egiten du)
 - zenbaki oso sekuentzia → zatitze-indizeak (puntuak)
 - `axis` → zein ardatzetan zatitu

```
In [46]: a = np.arange(42).reshape((6,7))
upper,center,lower = np.split(a,3)
print(f'{a=}\n{upper=}\n{center=}\n{lower=}')
```

```
a=array([[ 0,  1,  2,  3,  4,  5,  6],
        [ 7,  8,  9, 10, 11, 12, 13],
        [14, 15, 16, 17, 18, 19, 20],
        [21, 22, 23, 24, 25, 26, 27],
        [28, 29, 30, 31, 32, 33, 34],
        [35, 36, 37, 38, 39, 40, 41]])
upper=array([[ 0,  1,  2,  3,  4,  5,  6],
            [ 7,  8,  9, 10, 11, 12, 13]])
center=array([[14, 15, 16, 17, 18, 19, 20],
            [21, 22, 23, 24, 25, 26, 27]])
lower=array([[28, 29, 30, 31, 32, 33, 34],
            [35, 36, 37, 38, 39, 40, 41]])
```

```
In [47]: a = np.arange(42).reshape((6,7))
# ERRORREA: arraya zatitzean ez da zatiketa berdina lortzen
#upper,lower = np.split(a,2,axis=1)
```

```
In [48]: a = np.arange(36).reshape((6,6))
upper,center,lower = np.split(a,(1,3))
print(f'{a=}\n{upper=}\n{center=}\n{lower=}')
```

```
a=array([[ 0,  1,  2,  3,  4,  5],
        [ 6,  7,  8,  9, 10, 11],
        [12, 13, 14, 15, 16, 17],
        [18, 19, 20, 21, 22, 23],
        [24, 25, 26, 27, 28, 29],
        [30, 31, 32, 33, 34, 35]])
upper=array([[0, 1, 2, 3, 4, 5]])
center=array([[ 6,  7,  8,  9, 10, 11],
            [12, 13, 14, 15, 16, 17]])
lower=array([[18, 19, 20, 21, 22, 23],
            [24, 25, 26, 27, 28, 29],
            [30, 31, 32, 33, 34, 35]])
```

```
In [49]: a = np.arange(0,14).reshape((2,7))
left,center,right = np.split(a,(1,3), axis=1)
print(f'{a=}\n{left=}\n{center=}\n{right=}')

a=array([[ 0,  1,  2,  3,  4,  5,  6],
        [ 7,  8,  9, 10, 11, 12, 13]])
left=array([[0],
           [7]])
center=array([[1, 2],
            [8, 9]])
right=array([[ 3,  4,  5,  6],
            [10, 11, 12, 13]])
```

- vsplit eta hspllit - 2 (edo gehiago) dimentsioko arrayak
 - vsplit(a,ii) == split(a, ii, axis=0)
 - hspllit(a,ii) == split(a, ii, axis=1)
- hspllit - 1 dimentsioko arrayak
 - hspllit(a,ii) == split(a, ii)

```
In [50]: a = np.arange(16).reshape((4,4))
b,c = np.vsplit(a,2)
d,e = np.split(a,2, axis=0)
print(f'{a=}\n{b=}\n{c=}\n{d=}\n{e=}')

a=array([[ 0,  1,  2,  3],
        [ 4,  5,  6,  7],
        [ 8,  9, 10, 11],
        [12, 13, 14, 15]])
b=array([[0, 1, 2, 3],
        [4, 5, 6, 7]])
c=array([[ 8,  9, 10, 11],
        [12, 13, 14, 15]])
d=array([[0, 1, 2, 3],
        [4, 5, 6, 7]])
e=array([[ 8,  9, 10, 11],
        [12, 13, 14, 15]])
```

```
In [51]: a = np.arange(16).reshape((4,4))
b,c = np.hsplit(a,2)
d,e = np.split(a,2, axis=1)
print(f'{a=}\n{b=}\n{c=}\n{d=}\n{e=}')
```

```
a=array([[ 0,  1,  2,  3],
        [ 4,  5,  6,  7],
        [ 8,  9, 10, 11],
        [12, 13, 14, 15]])
b=array([[ 0,  1],
        [ 4,  5],
        [ 8,  9],
        [12, 13]])
c=array([[ 2,  3],
        [ 6,  7],
        [10, 11],
        [14, 15]])
d=array([[ 0,  1],
        [ 4,  5],
        [ 8,  9],
        [12, 13]])
e=array([[ 2,  3],
        [ 6,  7],
        [10, 11],
        [14, 15]])
```

```
In [52]: a = np.arange(16)
b,c = np.hsplit(a,2)
d,e = np.split(a,2)
print(f'{a=}\n{b=}\n{c=}\n{d=}\n{e=}')
```

```
a=array([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14, 15])
b=array([0, 1, 2, 3, 4, 5, 6, 7])
c=array([ 8,  9, 10, 11, 12, 13, 14, 15])
d=array([0, 1, 2, 3, 4, 5, 6, 7])
e=array([ 8,  9, 10, 11, 12, 13, 14, 15])
```

UFunc-ak - Funtzio unibertsalak

Arrayetan elementuz elementuko eragiketak egiteko diseinatutako funtzioak. Ezaugarri nagusiak:

- **Abiadura eta eraginkortasuna:** elementu bakoitzean kalkulu *bera* egiteko arrayak begizta bidez (`for` , `while`) zeharkatu beharrean, optimizatutako funtzioak erabiltzen dira.
- **Broadcasting-a:** forma desberdin baina *bateragarria* duten arrayekin aritzeko aukera.
- **Type casting:** datu mota desberdinekin aritzeko aukera

Eragiketa aritmetikoak

```
In [53]: x = np.arange(8)
print("      x =", x)
print(" x + 5 =", x + 5)
print(" x - 5 =", x - 5)
print("      -x =", -x)
print(" x * 2 =", x * 2)
print(" x / 2 =", x / 2)
print("x // 2 =", x // 2)
print(" x % 2 =", x % 2)
print("x ** 2 =", x ** 2)
```

```
      x = [ 0  1  2  3  4  5  6  7]
x + 5 = [ 5  6  7  8  9 10 11 12]
x - 5 = [-5 -4 -3 -2 -1  0  1  2]
      -x = [ 0 -1 -2 -3 -4 -5 -6 -7]
x * 2 = [ 0  2  4  6  8 10 12 14]
x / 2 = [0.  0.5 1.  1.5 2.  2.5 3.  3.5]
x // 2 = [0 0 1 1 2 2 3 3]
x % 2 = [0 1 0 1 0 1 0 1]
x ** 2 = [ 0  1  4  9 16 25 36 49]
```

Eragile aritmetiko bakoitza, izatez, NumPy-n integratutako funtzio jakin baten estaldura bat (*wrapper*) da

Eragilea	Ufunc baliokidea	Deskribapena
+	np.add	Batuketa (e.g., $1 + 1 = 2$)
-	np.subtract	Kenketa (e.g., $3 - 2 = 1$)
-	np.negative	Ezeztapen unarioa (e.g., -2)
*	np.multiply	Biderketa (e.g., $2 * 3 = 6$)
/	np.divide	Zatiketa (e.g., $3 / 2 = 1.5$)
//	np.floor_divide	Beheranzko zatiketa osoa (e.g., $3 // 2 = 1$)
**	np.power	Berreketa (e.g., $2 ** 3 = 8$)
%	np.mod	Modulua/hondarra (e.g., $9 \% 4 = 1$)

Algebra linealeko eragiketak

Eragile aritmetikoek elementuz elementuko eragiketak adierazten dituzte:

```
In [54]: x = np.arange(9).reshape((3,3))
y = np.eye(3)
print(f'{x=}\n{y=}')
print(f'{x+y=}')
print(f'{x*y=}')

```

```
x=array([[0, 1, 2],
        [3, 4, 5],
        [6, 7, 8]])
y=array([[1., 0., 0.],
        [0., 1., 0.],
        [0., 0., 1.]])
x+y=array([[1., 1., 2.],
          [3., 5., 5.],
          [6., 7., 9.]])
x*y=array([[0., 0., 0.],
          [0., 4., 0.],
          [0., 0., 8.]])

```

Algebra linealeko eragiketak adierazten dituzten eragile bereziak daude:

- $A @ B$ edo `np.matmul(A,B)` → Matrizen biderketa

```
In [55]: x = np.arange(9).reshape((3,3))
y = np.eye(3)
print(f'{x=}\n{y=}')
print(f'{x@y=}')
print(f'{np.matmul(x,y)=}')

```

```
x=array([[0, 1, 2],
        [3, 4, 5],
        [6, 7, 8]])
y=array([[1., 0., 0.],
        [0., 1., 0.],
        [0., 0., 1.]])
x@y=array([[0., 1., 2.],
          [3., 4., 5.],
          [6., 7., 8.]])
np.matmul(x,y)=array([[0., 1., 2.],
                    [3., 4., 5.],
                    [6., 7., 8.]])

```

Algebra linealeko beste eragiketa batzuk:

"Eragilea"	Ufunc baliokidea	Deskribapena
A @ B	np.matmul(A,B)	Biderketa
A.T	np.transpose(A)	Iraulteta (bista)
	np.linalg.inv(A)	Alderantzizkoa
	np.linalg.pinv(A)	Sasi-alderantzizkoa
	np.trace(A)	Aztarna
	np.dot(u, v)	Biderkadura eskalarra
	np.cross(u, v)	Biderkadura bektoriala
	np.inner(u, v)	Barne-biderkadura
	np.outer(u, v)	Kanpo-biderkadura

Funtzio trigonometrikoak

```
In [56]: theta = np.linspace(0, np.pi, 4)
print(f'{theta=}')
print(f'{np.sin(theta)=}')
print(f'{np.cos(theta)=}')
print(f'{np.tan(theta)=}')

print("-----")
x = [-1, 0, 1]
print(f'{np.arcsin(x)=}')
print(f'{np.arccos(x)=}')
print(f'{np.arctan(x)=}')

theta=array([0.          , 1.04719755, 2.0943951 , 3.14159265])
np.sin(theta)=array([0.00000000e+00, 8.66025404e-01, 8.66025404e-01, 1.224
64680e-16])
np.cos(theta)=array([ 1. ,  0.5, -0.5, -1. ])
np.tan(theta)=array([ 0.00000000e+00,  1.73205081e+00, -1.73205081e+00, -
1.22464680e-16])
-----
np.arcsin(x)=array([-1.57079633,  0.          ,  1.57079633])
np.arccos(x)=array([ 3.14159265,  1.57079633,  0.          ])
np.arctan(x)=array([-0.78539816,  0.          ,  0.78539816])
```

Funtzio matematiko gehiago

- abs() , np.absolute() or np.abs()

```
In [57]: x = np.array([-2, -1, 0, 1, 2])
print(f'{abs(x)=}')
print(f'{np.absolute(x)=}')
print(f'{np.abs(x)=}')

abs(x)=array([2, 1, 0, 1, 2])
np.absolute(x)=array([2, 1, 0, 1, 2])
np.abs(x)=array([2, 1, 0, 1, 2])
```

- `np.exp()` , `np.exp2()` eta `np.power()`

```
In [58]: x = np.arange(3)
print("      x =", x)
print("    e^x =", np.exp(x))
print("    2^x =", np.exp2(x))
print("    3^x =", np.power(3, x))      # int motari eusten dio
print("3.14^x =", np.power(3.14, x))

x = [0 1 2]
e^x = [1.          2.71828183  7.3890561 ]
2^x = [1.  2.  4.]
3^x = [1 3 9]
3.14^x = [1.    3.14   9.8596]
```

- `np.log()` , `np.log2()` eta `np.log10()`

```
In [59]: x = np.array([1,2,np.e,10])
print("      x =", x)
print(" log2(x) =", np.log2(x))
print("  log(x) =", np.log(x))
print("log10(x) =", np.log10(x))

x = [ 1.          2.          2.71828183 10.          ]
log2(x) = [0.          1.          1.44269504 3.32192809]
log(x) = [0.          0.69314718 1.          2.30258509]
log10(x) = [0.          0.30103   0.43429448 1.          ]
```

- `np.sqrt()`

```
In [60]: x = np.linspace(0,100,5)
print("      x =", x)
print(" sqrt(x) =", np.sqrt(x))

x = [ 0.  25.  50.  75. 100.]
sqrt(x) = [ 0.          5.          7.07106781  8.66025404 10.          ]
```

- `np.round()`

```
In [61]: x = np.linspace(0,9.3,7)
print("      x =", x)
print("round(x) =", np.round(x))

      x = [0.   1.55 3.1  4.65 6.2  7.75 9.3 ]
round(x) = [0.  2.  3.  5.  6.  8.  9.]
```

Agregazio-funtzioak

- Array bat hartu eta eskalar bakar bat (edo array *txikiago* bat) itzultzen du
- `sum`, `median`, `max`, `min`, ...

- `np.sum(a)` edo `a.sum()`
- **EZ ERABILI** python-eko `sum` built-in funtzioa

```
In [62]: n = 100_000
x = np.arange(1,n+1)
sum(x) == np.sum(x) == x.sum() == n*(n+1)/2
```

Out[62]: True

```
In [63]: %timeit sum(x)
          %timeit np.sum(x)
          %timeit x.sum()
```

5.62 ms ± 94 µs per loop (mean ± std. dev. of 7 runs, 100 loops each)
 15.5 µs ± 213 ns per loop (mean ± std. dev. of 7 runs, 100,000 loops each)
 14.6 µs ± 511 ns per loop (mean ± std. dev. of 7 runs, 100,000 loops each)

- `np.min(a)` or `a.min()` eta `np.max(a)` or `a.max()`
- **EZ ERABILI** python-eko `min` edo `max` built-in funtzioak

```
In [64]: x = np.random.rand(100000)
min(x) == np.min(x) == x.min()
```

Out[64]: True


```
In [65]: %timeit min(x)
          %timeit np.min(x)
          %timeit x.min()
```

4.42 ms $\pm$ 70.8 μ s per loop (mean $\pm$ std. dev. of 7 runs, 100 loops each)
 16.1 μ s $\pm$ 271 ns per loop (mean $\pm$ std. dev. of 7 runs, 100,000 loops each)
 14.8 μ s $\pm$ 183 ns per loop (mean $\pm$ std. dev. of 7 runs, 100,000 loops each)

Agregazio-funtzio gehiago

Funtzio izena	NaN-save bertsioa	Deskribapena
np.sum	np.nansum	Elementuen batura kalkulatu
np.prod	np.nanprod	Elementuen biderkadura kalkulatu
np.mean	np.nanmean	Elementuen batezbestekoa kalkulatu
np.std	np.nanstd	Desbideratze estandarra kalkulatu
np.var	np.nanvar	Bariantza kalkulatu
np.min	np.nanmin	Balio minimoa aurkitu
np.max	np.nanmax	Balio maximoa aurkitu
np.argmin	np.nanargmin	Balio minimoaren indizea aurkitu
np.argmax	np.nanargmax	Balio maximoaren indizea aurkitu
np.median	np.nanmedian	Elementuen mediana kalkulatu
np.percentile	np.nanpercentile	Elementuen mailan (<i>rank</i>) oinarritutako estatistikak kalkulatu
np.any	N/A	Elementuren bat egiazkoa den ebaluatu
np.all	N/A	Elementu guztiak egiazkoak diren ebaluatu

Dimentsio anitzeko agregazioak

- Lehenespenez, agregazioa array osohauen formakn gainean egiten da.
- Argumentu gehigarri batek (*axis*) agregazioa zein ardatzetan kalkulatu den adierazten du

```
In [66]: a = np.arange(20).reshape((4,5))
          print(f'{a=}')
          print(f'{a.sum(=)}')
          print(f'{a.sum(axis=0)=}')
          print(f'{a.sum(axis=1)=}')

a=array([[ 0,  1,  2,  3,  4],
         [ 5,  6,  7,  8,  9],
         [10, 11, 12, 13, 14],
         [15, 16, 17, 18, 19]])
a.sum(=)190
a.sum(axis=0)=array([30, 34, 38, 42, 46])
a.sum(axis=1)=array([10, 35, 60, 85])
```

Broadcasting-a

Tamaina desberdineko arrayetan ufunc bitarrak (adibidez, batuketa, kenketa, biderketa eta abar) aplikatzeko gaitasuna.

```
In [67]: a = np.arange(10)
b = a[::-1]
print(f'{ a = }')
print(f'{ b = }')
print(f'{a+b = }')
print(f'{a+3 = }') # broadcasting-a

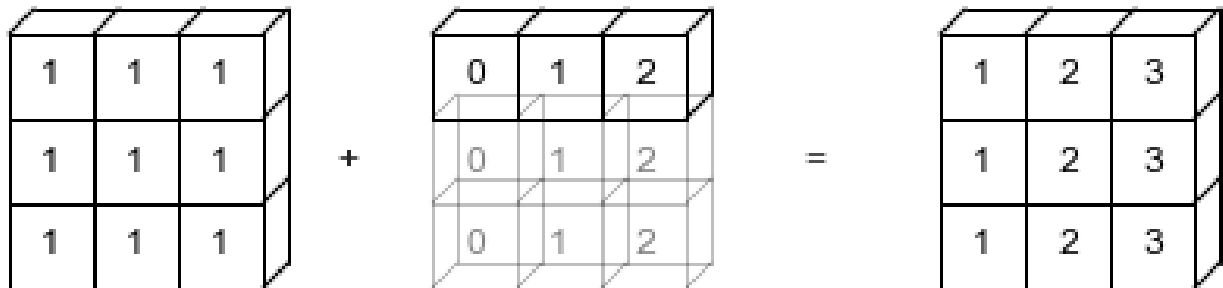
a = array([0, 1, 2, 3, 4, 5, 6, 7, 8, 9])
b = array([9, 8, 7, 6, 5, 4, 3, 2, 1, 0])
a+b = array([9, 9, 9, 9, 9, 9, 9, 9, 9, 9])
a+3 = array([ 3, 4, 5, 6, 7, 8, 9, 10, 11, 12])
```

Broadcasting-a - Adibide bisuala

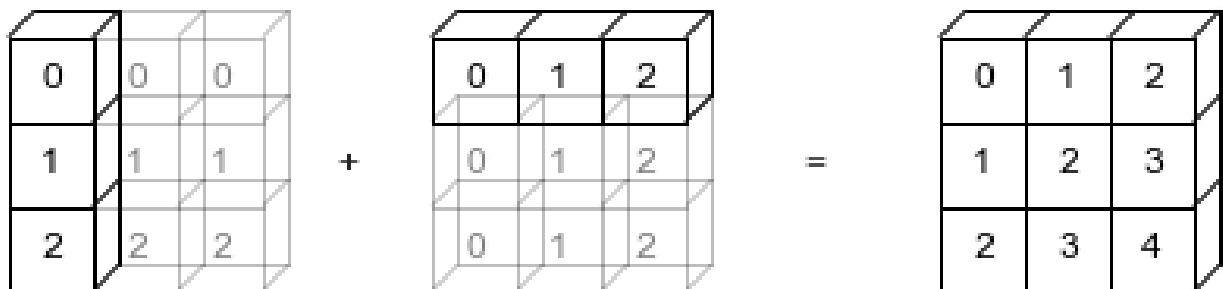
`np.arange(3) + 5`



`np.ones((3, 3)) + np.arange(3)`



`np.arange(3).reshape((3, 1)) + np.arange(3)`



```
In [68]: a = np.array([10,20,30,40,50])
a2 = a.reshape((1,5))
b = np.arange(10).reshape(2,5)
c = np.array([[ -1],[ -10]])
print(f'{ a = } {a2 = }')
print(f'{ b = }')
print(f'{ c = }')
print(f'{ a+a = }')
print(f'{ a+3 = }') # broadcasting-a (hedapena)
print(f'{ a+b = }') # broadcasting-a (hedapena)
print(f'{ a+c = }') # broadcasting-a (hedapena)
print(f'{ a2+c = }') # broadcasting-a (hedapena)
```

```
a = array([10, 20, 30, 40, 50]) a2 = array([[10, 20, 30, 40, 50]])
b = array([[0, 1, 2, 3, 4],
          [5, 6, 7, 8, 9]])
c = array([[ -1],
          [-10]])
a+a = array([ 20, 40, 60, 80, 100])
a+3 = array([13, 23, 33, 43, 53])
a+b = array([[10, 21, 32, 43, 54],
          [15, 26, 37, 48, 59]])
a+c = array([[ 9, 19, 29, 39, 49],
          [ 0, 10, 20, 30, 40]])
a2+c = array([[ 9, 19, 29, 39, 49],
          [ 0, 10, 20, 30, 40]])
```

Broadcasting-aren arauak

$(d_1^x, \dots, d_n^x)$ eta $(d_1^y, \dots, d_m^y)$ dimentsioak dituzten x eta y arrayak emanda

1. $n \neq m$ bada, dimentsio gutxien dituenaren *shape*-a ezkerretik 1 balioekin betetzen da.

- $n < m \rightarrow (1_1, \dots, 1_{m-n}, d_1^x, \dots, d_n^x)$
- $n > m \rightarrow (1_1, \dots, 1_{n-m}, d_1^y, \dots, d_m^y)$
- Betetzearen ondoren, x eta y -ren *shape*-ak $(d_1^x, \dots, d_k^x)$ eta $(d_1^y, \dots, d_k^y)$ dira, non $k = \max(n, m)$

2. Bi arrayen formak dimentsioren batean bat ez badatoz, dimentsio horretan 1 tamaina duen arraya hedatu (errepikatu) egiten da bestearen tamainarekin bat etor dadin.

- Hedapenaren ondoren, $\forall i \ d_i^x = 1 \iff d_i^y = 1$

3. Dimentsioren batean tamainak bat ez badatoz ($\exists i \text{ non } d_i^x \neq d_i^y$), errore bat sortzen da

- Errorerik sortzen ez bada, x eta y -ren *shape*-a $(d_1, \dots, d_k)$ da

4. Dimentsio guztiak bat datoz, emaitzaren *shape*-a $(d_1, \dots, d_k)$ delarik

Adibidea 1

```
In [69]: M = np.ones((2, 3))
a = np.arange(3)
```

- `M.shape == (2, 3)`
- `a.shape == (3,)`
- 1. araua: betetzea (dimentsio kopurua handitu)
 - `M.shape → (2, 3)`
 - `a.shape → (1, 3)`
- 1. araua: hedatzea (`1 → j`)
 - `M.shape → (2, 3)`
 - `a.shape → (2, 3)`
- 1. araua: dimentsioak konprobatu → **OK**

```
In [70]: M + a
```

```
Out[70]: array([[1., 2., 3.],
               [1., 2., 3.]])
```

Adibidea 2

```
In [71]: a = np.arange(3).reshape((3, 1))
        b = np.arange(3)
```

- `M.shape == (3, 1)`
- `a.shape == (3,)`
- 1. araua: betetzea (dimentsio kopurua handitu)
 - `M.shape → (3, 1)`
 - `a.shape → (1, 3)`
- 1. araua: hedatzea (`1 → j`)
 - `M.shape → (3, 3)`
 - `a.shape → (3, 3)`
- 1. araua: dimentsioak konprobatu → **OK**

```
In [72]: a + b
```

```
Out[72]: array([[0, 1, 2],
               [1, 2, 3],
               [2, 3, 4]])
```

Adibidea 3

```
In [73]: M = np.ones((3, 2))
        a = np.arange(3)
```

- `M.shape == (3, 2)`
- `a.shape == (3,)`
- 1. araua: betetzea (dimentsio kopurua handitu)
 - `M.shape → (3, 2)`
 - `a.shape → (1, 3)`
- 1. araua: hedatzea (`1 → j`)
 - `M.shape → (3, 2)`
 - `a.shape → (3, 3)`
- 1. araua: dimentsioak konprobatu → **ERROREA**

```
In [74]: # ERROREA: ezin izan da broadcasting-a egin
#M + a
```

Boolearrak eta maskaratzea (*masking*)

- Konparazio-eragileek/ufunc-ek elementuz elementuko boolearrak lortzen dituzte, array boolearrak sortuz
- Array boolearrak (**maskarak**) arrayetan kalkulu selektiboak egiteko erabil daitezke.
- Optimizatutako eragiketa bektorizatuak

```
In [75]: x = np.arange(7)
mask = x%3==0
print(f'{x = }')
print(f'{mask = }')
print(f'{x[mask] = }')
print(f'{x[x%3==0] = }')
x[x%3==0] = -10
print(f'{x = }')

x = array([0, 1, 2, 3, 4, 5, 6])
mask = array([ True, False, False,  True, False, False,  True])
x[mask] = array([0, 3, 6])
x[x%3==0] = array([0, 3, 6])
x = array([-10,  1,  2, -10,  4,  5, -10])
```

`y = x[mask]` beti da **kopia** bat, ez bista bat

```
In [76]: x = np.arange(7)
y = x[x%3==0]
print(f'{x = }')
print(f'{y = }')
y[0] = -10
print(f'{x = }')
print(f'{y = }')

x = array([0, 1, 2, 3, 4, 5, 6])
y = array([0, 3, 6])
x = array([0, 1, 2, 3, 4, 5, 6])
y = array([-10,  3,  6])
```

Konparazio-eragile bakoitza, izatez, NumPy-n integratutako funtzio jakin baten estaldura (*wrapper*) bat da

Eragilea	Ufunc baliokidea
>	np.greater
<	np.less
>=	np.greater_equal
<=	np.less_equal
==	np.equal
!=	np.not_equal

Python-en bit mailako eragile logikoak (& , | , ^ , eta ~)

- Elementuz elementuko eragiketa boolearrak (eragile aritmetikoen antzekoak)

```
In [77]: x = np.arange(7)
mask = (x%3==0) & (x<5)
print(f'{x = }')
print(f'{mask = }')
print(f'{x[mask] = }')
print(f'{x[(x%3==0)&(x<5)] = }')

x = array([0, 1, 2, 3, 4, 5, 6])
mask = array([ True, False, False,  True, False, False, False])
x[mask] = array([0, 3])
x[(x%3==0)&(x<5)] = array([0, 3])
```

Eragile logiko bakoitza, izatez, NumPy-n integratutako funtzio jakin baten estaldura (*wrapper*) bat da:

Eragilea	Ufunc baliokidea
&	np.bitwise_and
	np.bitwise_or
^	np.bitwise_xor
~	np.bitwise_not

- np.any(a) edo a.any() → True | False agregatua
- np.all(a) edo a.all() → True | False agregatua
- axis parametroa → ardatz jakin baten gaineko agregazioa

```
In [78]: x = np.arange(10).reshape((2,5))
print(f'{x=}')
print(f'{x>7=}')
print(f'{np.any(x>7)=}')
print(f'{(x>7).any()=}')
print(f'{np.all(x>7)=}')
print(f'{(x>7).all()=}')
print(f'{np.any(x>7, axis=0)=}')
print(f'{np.any(x>7, axis=1)=}')

x=array([[0, 1, 2, 3, 4],
        [5, 6, 7, 8, 9]])
x>7=array([[False, False, False, False, False],
        [False, False, False,  True,  True]])
np.any(x>7)=True
(x>7).any()=True
np.all(x>7)=False
(x>7).all()=False
np.any(x>7, axis=0)=array([False, False, False,  True,  True])
np.any(x>7, axis=1)=array([False,  True])
```

Array boolearrak batu daitezke

- True==1 , False==0
- np.sum(a) edo a.sum() → True | False agregatua
- axis parametroa → ardatz jakin baten gaineko agregazioa
- **EZ ERABILI** Pythoneko sum built-in funtzioa

```
In [79]: x = np.arange(10).reshape((2,5))
print(f'{x=}')
print(f'{x>7=}')
print(f'{np.sum(x>7)=}')
print(f'{(x>7).sum()=}')
print(f'{np.sum(x>7, axis=0)=}')
print(f'{np.sum(x>7, axis=1)=}')

x=array([[0, 1, 2, 3, 4],
        [5, 6, 7, 8, 9]])
x>7=array([[False, False, False, False, False],
        [False, False, False,  True,  True]])
np.sum(x>7)=2
(x>7).sum()=2
np.sum(x>7, axis=0)=array([0, 0, 0, 1, 1])
np.sum(x>7, axis=1)=array([0, 2])
```

Fancy Indexing

- Arrayetako elementu anitzera iristeko, indize arrayak erabil daitezke
- Datuen **kopia** bat itzultzen du (slicing-aren *view*-aren ordez)

```
In [80]: a = np.random.randint(100,size=(10,))
ii1 = [1, 8, 3, 5]
ii2 = np.array([1, 8, 3, 5])
print(f'{a = }')
print(f'{a[ii1] = }')
print(f'{a[ii2] = }')

a = array([49, 42,  1, 85, 58, 90, 57, 33, 63, 92])
a[ii1] = array([42, 63, 85, 90])
a[ii2] = array([42, 63, 85, 90])
```

Fancy Indexing-a dimentsio anitzeko arrayekin erabil daiteke

- Arrayetako elementu anitzera iristeko, indize arrayak erabil daitezke
- **Hautaketa sekuentziala**: dimentsio bakarreko array bat itzultzen du

```
In [81]: a = np.arange(15).reshape((3,5))
ii = [1, 0, 2, 2]
jj = [1, 4, 3, 2]
print(f'{a = }')
print(f'{a[ii,jj] = }')

a = array([[ 0,  1,  2,  3,  4],
           [ 5,  6,  7,  8,  9],
           [10, 11, 12, 13, 14]])
a[ii,jj] = array([ 6,  4, 13, 12])
```

Fancy Indexing-a indize soilekin konbina daiteke

- *Indizeen broadcasting* moduko bat

```
In [82]: a = np.arange(16).reshape((4,4))
ii = [1, 3, 2]
print(f'{a = }')
print(f'{a[1,ii] = }')
print(f'{a[ii,2] = }')

a = array([[ 0,  1,  2,  3],
           [ 4,  5,  6,  7],
           [ 8,  9, 10, 11],
           [12, 13, 14, 15]])
a[1,ii] = array([5, 7, 6])
a[ii,2] = array([ 6, 14, 10])
```

Azpiarrayak lor daitezke

- *Fancy Indexing*-a lehen dimentsioan soilik erabiltzean → errenkadak
- *slicing*-arekin konbinatuz → zutabeak


```
In [83]: a = np.arange(15).reshape((3,5))
ii = [2, 0]
jj = [1, 4, 3]
print(f'{a = }')
print(f'{a[ii] = }')
print(f'{a[ii,:] = }')
print(f'{a[:,jj] = }')

a = array([[ 0,  1,  2,  3,  4],
           [ 5,  6,  7,  8,  9],
           [10, 11, 12, 13, 14]])
a[ii] = array([[10, 11, 12, 13, 14],
               [ 0,  1,  2,  3,  4]])
a[ii,:] = array([[10, 11, 12, 13, 14],
                 [ 0,  1,  2,  3,  4]])
a[:,jj] = array([[ 1,  4,  3],
                 [ 6,  9,  8],
                 [11, 14, 13]])
```